AI Email Support Agent

Camunda 8.10 Blueprint → CP4BA 26.0.0 Map

Design-review pack: per-node tally, target architecture, runbook, AI-capability touchpoints, and component tally — with the confirmed stub provider seam.

Scope & grounding

Ground truth: the per-node and component migration tally.

Docs: IBM CP4BA 26.0.0 pages we fetched (cluster prep, client prep, external DB config, CR authoring, Model Gateway, watsonx.ai LWE, MCP servers).

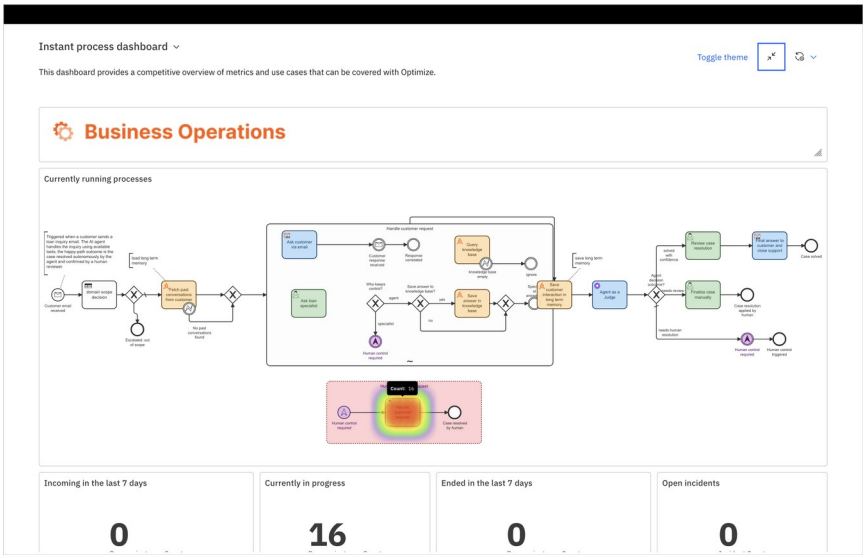
Inference flags: rows marked “verify” are reasonable reads of the docs, not enumerated facts — confirm against your interim-fix level.

Contents

Agent Blueprint (Long Term Memory) running on Camunda 8.10 docker compose

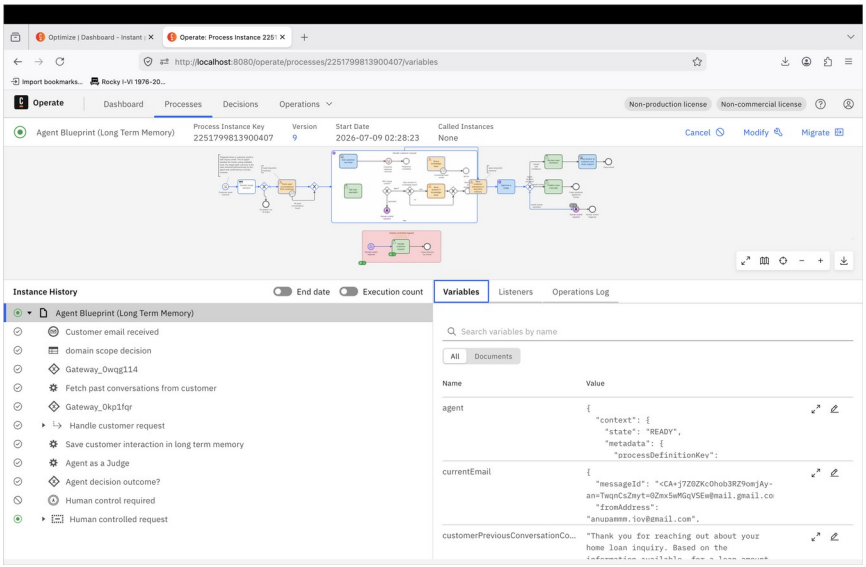
Two views of the live system that ground this document, so a reviewer can map what the tally describes against what actually runs.

Camunda Optimize — the instant process dashboard: the full agent BPMN with live metrics (16 instances in progress), the ad-hoc “Handle customer request” subprocess, the Agent-as-a-Judge activity, and the human-controlled resolution paths.



→ In CP4BA: the same agent is re-platformed onto BAW Workflow Runtime. It ingests the inbound support email, checks scope against the ODM guardrail, retrieves prior conversations from long-term memory, runs its tool-calling loop (knowledge base, loan specialist) and the Agent-as-a-Judge evaluation, then either closes the case autonomously or routes it to a human — with every step's KPIs surfaced through the BPM event emitter into Business Automation Insights.

Camunda Operate — a single running instance (version 9): the instance-history token trail through each activity, and the live runtime variables including the agent state, the customer email, and the retrieved prior-conversation context.



→ In CP4BA: this same instance-level visibility is provided by BAW, where each activity's state and variables are inspected in the runtime. The agent's model calls (generation and the judge) are routed through the Model Gateway to a governed provider, and the long-term-memory embeddings and retrieval are served in-cluster by watsonx.ai LWE — keeping the customer email and conversation data inside the OpenShift boundary.

0. The confirmed provider seam

The POC agent loop did not call a real model. It called `stub_llm_server.py` — a Flask server exposing an OpenAI-compatible `/v1/chat/completions` endpoint on `0.0.0.0:9999`. It is a two-turn state machine: turn 0 inspects the incoming `tools` array, finds the tool whose name/description contains "knowledge", and returns a `tool_call`; turn 1 returns a canned home-loan answer and stops. Token counts in the `usage` field are hardcoded fakes.

This matters because the stub is a placeholder for exactly the seam the **Model Gateway** fills in production: both present an OpenAI-compatible chat-completions contract. In the POC the connector pointed at a stub that faked the model; in prod the BAW GenAI/agent activity points at the Model Gateway, which routes to a real model (watsonx.ai LWE or an external provider). **Same contract, real backend — so the connector-integration work transfers.**

> **What the stub did NOT prove (carry into design review)**
>
> The stub always picks the single "knowledge" tool and then stops. It never exercises the multi-tool decision space in your BPMN — Ask loan specialist, the Who-keeps-control (agent vs specialist) branch, Save-answer-to-KB, and critically the Agent-as-a-Judge step.
>
> It therefore proved the plumbing (connector ↔ OpenAI contract ↔ one tool round-trip), not the agent's judgment under a real LLM. The judge behavior is unproven end-to-end.
>
> Hardcoded token counts mean the POC produced no real usage telemetry — that only appears once a real model behind the Gateway is generating tokens.

1. Per-node POC → CP4BA prod tally

Every element read from the BPMN, left-to-right, mapped to its CP4BA production home. The "AI capability" column marks where Model Gateway (MG), watsonx.ai LWE, or MCP servers touch that node.

BPMN node	POC implementation	CP4BA prod equivalent	AI capability	Change / risk
Customer email received (message start)	Camunda message start; mail poller	BAW message/UCA start event; inbound email listener	—	Rewire trigger to BAW inbound channel
domain scope decision (business rule)	DMN domain-scope-guardrail.dmn (FEEL, FIRST hit)	ODM decision service (or DMN in BAW)	**MCP (ODM, opt.)**	Author guardrail in ODM; agent can call it as a tool
Escalated: out of scope (terminate)	Terminate end event	BAW terminate / end state	—	Direct equivalent
Fetch past conversations from customer (agent/tool)	Job worker + ES query; "load long term memory"	BAW service activity → retrieval store	**LWE (embeddings)**	RAG retrieval; granite embeddings on LWE
No past conversations found (gateway)	XOR gateway	BAW gateway	—	Direct equivalent
Ask customer via email (send)	Send task	BAW send / email integration	—	Map to BAW email service
Customer response received / Response correlated (receive)	Message receive + correlation	BAW receive event + correlation key	—	Preserve correlation id design
Ask loan specialist	Tool/task (never hit by stub)	BAW human/service task or agent tool	**MCP (Workflow)**	Unproven in POC — test under real

BPMN node	POC implementation	CP4BA prod equivalent	AI capability	Change / risk
				model
Query knowledge base (agent tool)	The ONE tool the stub called; ES dense-vector	BAW agent tool → vector KB	**LWE + MCP**	Only proven tool path; embeddings via LWE
knowledge base empty / ignore (gateways)	XOR branches	BAW gateways	—	Direct equivalent
Who keeps control? (agent vs specialist)	XOR gateway (never exercised by stub)	BAW gateway	—	Unproven branch — validate agent decision
Save answer to knowledge base? → Save answer in KB	Decision + write task (not hit by stub)	BAW service task → KB write	**MCP (Content, opt.)**	Unproven; wire write path
Human control required (boundary/escape)	Boundary escalation → human	BAW human task escalation	—	Map to BAW work management
Save customer interaction in long term memory	Job worker; “save long term memory”; ES write	BAW service task → retrieval store write	**LWE (embeddings)**	Persist + embed for future RAG
Agent as a Judge (GenAI task)	Blue GenAI task; would have called stub — NEVER simulated	BAW GenAI “Agent as a Judge” activity	**MG → LWE**	Highest-risk gap: judge unproven under real LLM
Agent decision outcome? (gateway)	XOR on judge output	BAW gateway on GenAI result	—	Depends on real judge output shape
solved with confidence → Review case resolution → Final answer & close → Case solved	Happy path (stub’s canned answer landed here)	BAW review + close tasks	—	Proven only with fake answer
needs review → Finalize case manually → Case resolution applied by human	Human path	BAW human task	—	Direct equivalent
needs human resolution → Human control required → Human control triggered	Escalation path	BAW human task escalation	—	Direct equivalent
Human controlled request (event subprocess) → Handle customer request → Case resolved by human	Event subprocess (heatmap-highlighted)	BAW event subprocess	—	Direct equivalent
(LLM backend)	stub_llm_server.py — fake OpenAI endpoint :9999	Real model served in-cluster	**MG + LWE**	The core substitution — see Part 5

2. CP4BA target architecture

The production topology re-draws the POC as layers inside the OpenShift boundary. The diagram is rendered inline in the chat alongside this document; the layer description below is the authoritative reference.

Layers, top to bottom

- **Process layer —** BAW Workflow Runtime on CP4BA runs the migrated process (start → scope → fetch memory → ad-hoc agent subprocess → save memory → Agent-as-a-Judge → resolution routing → human tasks). Backed by BPMDB/PDWDB.
- **Decision layer —** ODM hosts the domain-scope guardrail; exposed to the agent via the ODM Decision MCP server.
- **Agent-tools layer (MCP) —** Workflow Runtime MCP Server (find/start/manage processes & cases under OAuth end-user identity), plus optional Content Core and ODM MCP servers. This is the productionized version of the MCP pattern you already validated on 8.10.
- **AI-gateway layer —** Model Gateway presents the OpenAI-compatible endpoint the BAW GenAI/agent activity calls (replacing :9999). Holds provider/model aliases; owns its external PostgreSQL and (at scale) Redis.
- **Model-serving layer —** watsonx.ai LWE on IBM Software Hub 5.4.x, GPU-backed, serves the generation model (agent + judge) and the granite embedding model (RAG). Alternative: external watsonx.ai/OpenAI/Bedrock fronted by the Gateway.
- **Data layer —** BAW DB topology (BPMDB, CMNDB, PDWDB, BASDB, AEDB, ICNDB, Zen, BTS) + the separate Model Gateway PostgreSQL + the vector KB store (retrieval / long-term memory).
- **Observability —** two planes: BPM event emitter → Business Automation Insights (process KPIs) and Model Gateway metrics (model/token/routing). These are complementary, not interchangeable.

3. Migration runbook (this process to prod)

Sequenced so each step's prerequisites are satisfied by the prior ones. Phases mirror the CP4BA install docs plus the three AI add-ons.

Phase A — Platform & base BAW

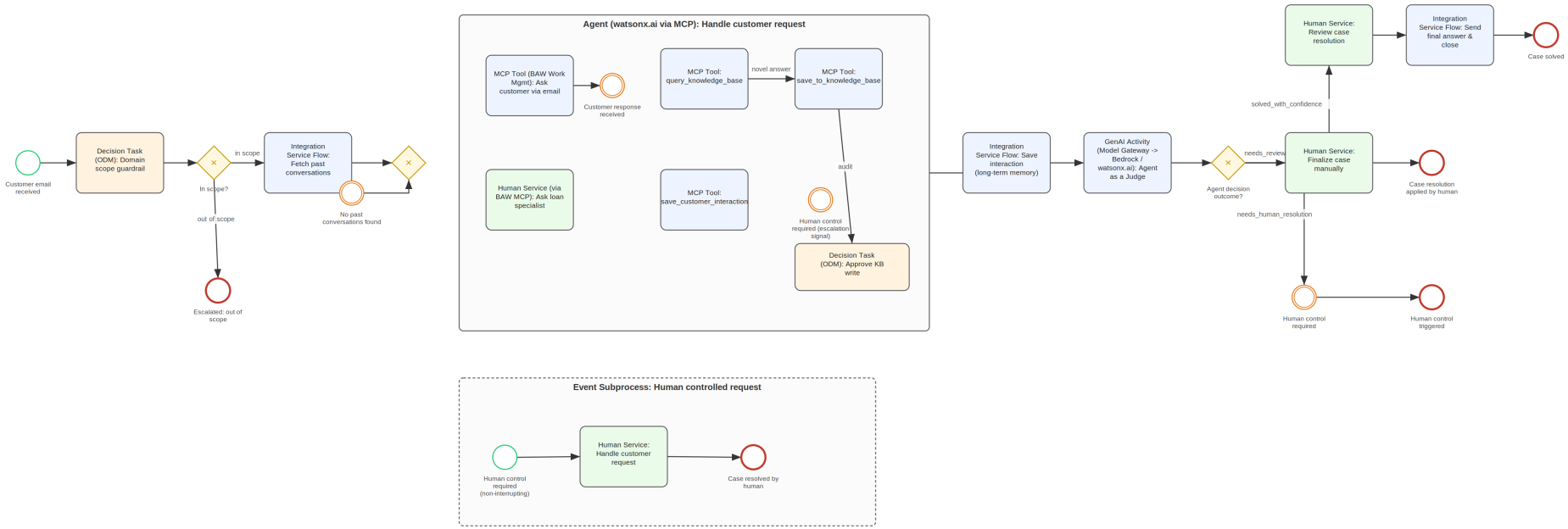
1. Prepare the OpenShift cluster: entitlement key, dynamic + block storage classes, secrets, admin/non-admin users, NTP sync, capacity for BAW + AI add-ons.
2. Prepare the admin client: Bash 4+, oc/kubectl, Podman, clone cert-kubernetes.
3. Provision external DBs and wire JDBC/datasource XML for the BAW topology (BPMDB, CMNDB, PDWDB, BASDB, AEDB, ICNDB, Zen, BTS).
4. Author the CR: shared_configuration (license, platform, sizing, storage classes, image pull secrets, TLS, LDAP, datasources) + baw_configuration (PVCs, secrets, datasources).
5. Validate YAML (yq / YAML Lint; no <Required> placeholders left), then apply the CR; operator reconciles BAW Workflow Runtime.
6. Migrate the process: import/rebuild the BPMN, port the job workers to BAW service activities, wire email in/out, human tasks. **Target: the full flow runs with human review, no GenAI yet.**

Phase B — Model serving (watsonx.ai LWE)

7. Install IBM Software Hub 5.4.x; confirm it is running.
8. Ensure GPU-capable worker nodes meeting Software Hub hardware/GPU requirements (biggest net-new infra ask).

2a. Target design diagram (BAW 26.x)

BPMN mock with BAW-faithful naming. This is a design mock for review, not a BAW artifact — BAW Process Designer does not import BPMN 2.0 XML. The naming is the migration contract; the pictur



Four regions:

A — Domain scope guardrail: Decision Task (ODM), kept deterministic and auditable rather than delegated to a model.

B — Handle customer request: Agent (watsonx.ai via MCP) — ad-hoc, agent-driven; KB/memory ops are MCP tools the agent selects at runtime. The knowledgeBaseDecision flag is gone; empty retrieval is a normal tool result; KB writes pass a Decision Task (ODM): Approve KB write audit gate.

C — Judge & route: GenAI Activity (Model Gateway → Bedrock / watsonx.ai): Agent as a Judge → deterministic gateway on the structured verdict (solved_with_confidence / needs_review / needs_human_resolution).

D — Human controlled request: event subprocess with plain BAW human services; no Case Builder, since every human step is bounded with a known successor.

9. cpd-cli login; create cluster-scoped resources; apply namespace permissions.
10. Batch-install `watsonx_ai` with `liteInstall: true`; add models to the LWE CR (`watsonxaiifm-cr`): granite-embedding-278m-multilingual (RAG) and a generation model (agent + judge). Air-gapped: mirror image groups with --skip_components.

Phase C — Model Gateway

11. Provision a SEPARATE external PostgreSQL (db e.g. modelgateway) + create model-gateway-postgres-external-secret. This is not a BAW schema.
12. Ensure block storage (Redis at medium/large scale) + file storage class; image pull secret in operator and instance namespaces.
13. Run `cp4a-model-gateway-deployment.sh` (from scripts/model-gateway) with --verify-only, then install; use --configure-providers to register the LWE (or external) provider → stored in model-gateway-provider-secret.
14. Enable generative AI in BAW through the Model Gateway; point the GenAI/agent activity at the Gateway endpoint (this is where :9999 is replaced).

Phase D — Agent tools (MCP) & the agent

15. Deploy the Workflow Runtime MCP Server; configure OAuth so it invokes under the end-user identity.
16. Expose the ODM guardrail via the ODM Decision MCP server (optional: Content Core MCP for KB writes).
17. Wire the ad-hoc agent subprocess to: call models via the Gateway, discover tools via MCP. Re-implement the multi-tool paths the stub never exercised.
18. Implement & test **Agent as a Judge** against the real model — the highest-risk item, since the stub never simulated it.

Phase E — Observability & cutover

19. Enable BPM event emitter → Business Automation Insights for process KPIs (replaces the process dimension of collect_metrics.py / Kibana).
20. Use Model Gateway metrics for model/token/routing telemetry (replaces the fake usage numbers the stub produced).
21. Shadow-run against POC results, validate guardrail + judge behavior, then cut over.

4. AI-capability touchpoint map (this flow)

Where each of the three capabilities attaches to specific nodes.

Capability	Touches these nodes	What it does here
Model Gateway	Agent-as-a-Judge; the ad-hoc agent loop; (the LLM backend seam)	The OpenAI-compatible endpoint the GenAI/agent activity calls instead of :9999. Provider abstraction, credentials, routing, per-call token metrics. Swap the judge/agent model by re-pointing the alias — no BPMN change.
watsonx.ai LWE	Agent-as-a-Judge; ad-hoc agent generation; Fetch/Save long-term memory (embeddings)	In-cluster, GPU-backed model serving. Generation model powers the agent + judge; granite embedding model powers RAG retrieval and long-term-memory writes — keeping customer PII inside the OpenShift boundary.
MCP servers	Fetch past conversations; Query KB; Ask loan specialist; Save answer in KB; domain scope decision (ODM); process control	Exposes BAW processes/cases (Workflow Runtime MCP), the ODM guardrail (Decision MCP), and content ops (Content Core MCP) as discoverable tools under OAuth. The

Capability	Touches these nodes	What it does here
		productionized form of the MCP pattern you validated on 8.10.

> **Swap-the-judge-model, concretely**
>
> The BAW GenAI activity is configured against a Model Gateway alias, not a provider URL. The Gateway holds the alias→provider+model mapping in its CR / provider-secret. To swap the judge model you re-run the deployment script with --configure-providers (or patch the CR) so the alias resolves to a different model. The BPMN activity keeps pointing at the same reference — hence "without touching BPMN".
>
> **Verify:** exactly how BAW's GenAI activity binds the model (alias vs pinned model name). If pinned, the swap is a config change on the activity — still not a process redesign.

5. Component-by-component migration tally

The full inventory now that the stub seam is confirmed — each POC artifact to its prod home and the capability that powers it.

POC component	CP4BA prod equivalent	Capability	Enhancement in prod
BPMN + job workers (trigger, fetch, save, human task)	BAW process app + service/human activities on Workflow Runtime	—	XA persistence, enterprise HA, durable BPMDB/PDWDB/ICNDB
`stub_llm_server.py` fake OpenAI endpoint :9999, canned answer, fake usage	Real model served in-cluster + OpenAI-compatible gateway endpoint	**MG + LWE**	Real backend behind the identical contract; connector work transfers unchanged
Ad-hoc agent loop (Camunda AI Agent connector → stub)	BAW GenAI activity → Model Gateway → real model	**MG → LWE**	Governed routing, provider failover, real token metrics
Agent-as-a-Judge activity	BAW GenAI "Agent as a Judge" activity	**MG → LWE**	Judge model routed + versioned via alias; swap without BPMN edit. NOTE: never simulated by the stub — test first
`domain-scope-guardrail.dmn` FEEL, FIRST hit	ODM decision service	**MCP (ODM)**	Enterprise authoring/versioning; callable by the agent as a tool
RAG: all-mpnet-base-v2 + ES dense-vector index	Embedding model + vector retrieval store	**LWE (granite embed)**	In-cluster embeddings; consistent model governance
`collect_metrics.py` bolt-on collector	Two native planes: BPM event emitter → BAI (process KPIs) + Model Gateway metrics (model/token)	**MG + BAI**	Native, split correctly by concern — no custom collector
MCP server proven on 8.10 (Podman/kind excursion)	Workflow Runtime MCP Server (+ ODM / Content MCP)	**MCP**	Different implementation, same pattern; OAuth end-user identity, central governance
Human task (final review)	BAW human task + Workspace / task list	—	Enterprise task routing, work management, audit

Open items to close before design review

- **Agent-as-a-Judge under a real model —** unproven; the stub never simulated it. Highest risk.

- **Multi-tool agent paths —** Ask loan specialist, Who-keeps-control branch, Save-answer-to-KB were never exercised (stub always took the single knowledge tool).
- **GPU + Software Hub for LWE —** substantial new dependency the 8.10 stack didn't need. If no GPU, front external watsonx.ai via the Gateway — but that reopens the data-egress question LWE was solving.
- **Model Gateway PostgreSQL —** a separate datasource beyond the BAW topology.

Prepared from the Optimize BPMN view + stub_llm_server.py + fetched IBM CP4BA 26.0.0 documentation. Rows flagged "verify" are doc-consistent inferences, not enumerated facts.